

Trabajo Práctico Unidad 7: Fundamentos y Técnicas de Diseño de Pruebas

Alumno: Ignacio Figueroa

Materia: Metodología de Sistemas I

Comisión: 7

Institución: Universidad Tecnológica Nacional (UTN) - Tecnicatura Universitaria en Programación

Parte 1: Fundamentos y Mentalidad del Tester

1. Definición de Testing de Software y el Costo Exponencial

El testing de software es el proceso de ejecutar y evaluar un sistema o sus componentes, de forma planificada y sistemática, para ver si cumple con los requisitos que nos pidió el cliente y encontrar fallas. No es solo un proceso para buscar bugs al final del proyecto, sino una forma de medir la calidad del producto durante todo el desarrollo.

La metáfora del "Costo Exponencial de un Defecto" explica que cuanto más tarde descubrimos un bug en el ciclo de vida del software, más caro y difícil es arreglarlo. Si encontramos un error de interpretación en la etapa de requisitos, resolverlo puede costar 1 peso (solo se aclara en una reunión). Si ese mismo error llega a la etapa de desarrollo, ya cuesta 100 pesos porque implica borrar y reescribir código. En la etapa de QA sube a 1.000 pesos por todo el circuito de reporte y re-testing. Pero si el bug llega a producción y lo encuentra el cliente, el costo se dispara a 10.000 pesos o más, ya que genera pérdida de datos, soporte de emergencia, parches apurados y una mala reputación para la empresa.

Por esto el testing es una inversión y no un gasto. Pagar un costo bajo control durante el desarrollo nos ahorra gastar fortunas en arreglar desastres cuando el sistema ya está publicado.

2. La Mentalidad del Tester (Formulario de Login)

A diferencia del desarrollador, que tiene una mentalidad constructiva y tiende a probar el "camino feliz" (que todo funcione cargando los datos bien), el tester tiene una mentalidad investigativa y escéptica. Su objetivo es ver bajo qué escenarios el sistema falla.

En un formulario de login, tres preguntas del tipo "¿Qué pasa si...?" que un tester se haría y que a un desarrollador se le podrían pasar de largo son:

- ¿Qué pasa si el usuario hace clic en el botón de 'Ingresar' muchas veces seguidas y muy rápido antes de que responda el servidor? (Para ver si se clava la app o si manda peticiones duplicadas).
- ¿Qué pasa si en el campo de contraseña meto caracteres especiales, emojis o directamente pego un fragmento de código o una consulta SQL? (Para probar la seguridad y que el sistema no rompa por la codificación).
- ¿Qué pasa si el usuario deja los campos vacíos o mete solamente espacios con la barra espaciadora e intenta iniciar sesión? (Para verificar si las validaciones del front-end y del back-end están funcionando bien).

3. Enfoques Fundamentales de Pruebas

Enfoque	Objetivo Principal	Ejemplo de Prueba Exclusiva
Caja Negra	Validar que el sistema haga lo que tiene que hacer a nivel funcional, mirando solo las entradas y salidas, sin saber cómo está programado por dentro.	Simular que un usuario final entra a la web, agrega un producto al carrito y completa la compra, verificando que la interfaz muestre el mensaje de éxito esperado.
Caja Blanca	Probar la estructura interna, la lógica del código fuente, los condicionales y los bucles para asegurar que la construcción sea robusta y eficiente.	Diseñar un caso de prueba unitario que fuerce a una función matemática a entrar específicamente por la rama del ELSE de un IF para medir la cobertura.

Enfoque	Objetivo Principal	Ejemplo de Prueba Exclusiva
Caja Gris	Evaluar las funciones del sistema teniendo un conocimiento parcial de la arquitectura interna o los componentes técnicos del backend.	Completar un formulario de registro en la interfaz (Caja Negra) y, si tira un error genérico, hacer una consulta SQL directa a la base de datos para revisar si el registro se guardó o no.

Parte 2: Técnicas de Diseño de Pruebas de Caja Negra

1. Requisito 1 (Campo "Edad": entre 18 y 65 años, inclusive)

Particiones de Equivalencia:

1. **P1 (Inválida)**: Valores menores a 18 (Ejemplo: 15).
2. **P2 (Válida)**: Valores en el rango de 18 a 65 inclusive (Ejemplo: 30).
3. **P3 (Inválida)**: Valores mayores a 65 (Ejemplo: 72).
4. **P4 (Inválida)**: Datos no numéricos o texto (Ejemplo: "veinte").
5. **P5 (Inválida)**: Dejar el campo completamente vacío.

Casos de prueba derivados de Valores Límite (BVA):

Para probar los bordes del rango (18 y 65) usamos los valores exactos y los que están justo al lado:

- Caso 1: Edad = 17 (Límite inferior - 1) → Resultado esperado: Rechazado / Inválido.
- Caso 2: Edad = 18 (Límite inferior exacto) → Resultado esperado: Aceptado / Válido.
- Caso 3: Edad = 19 (Límite inferior + 1) → Resultado esperado: Aceptado / Válido.
- Caso 4: Edad = 64 (Límite superior - 1) → Resultado esperado: Aceptado / Válido.
- Caso 5: Edad = 65 (Límite superior exacto) → Resultado esperado: Aceptado / Válido.
- Caso 6: Edad = 66 (Límite superior + 1) → Resultado esperado: Rechazado / Inválido.

2. Requisito 2 (Regla de Negocio: Descuento del 15% solo si es

Premium Y compra > \$50.000)

Tabla de Decisión:

Condiciones / Acciones	Regla 1	Regla 2	Regla 3	Regla 4
¿Cliente Premium?	Verdadero	Verdadero	Falso	Falso
¿Compra > \$50.000?	Verdadero	Falso	Verdadero	Falso
¿Aplicar Descuento 15%?	Sí	No	No	No

¿Por qué probar solo la Regla 1 es insuficiente?

Si probamos únicamente el caso donde se cumple todo (Regla 1), solo estamos verificando que el descuento funcione cuando las condiciones son correctas. Sin embargo, no estaríamos probando si el sistema restringe el beneficio cuando no corresponde. Podría pasar que el código tenga un error de lógica y le aplique el descuento a un cliente común (Regla 3) o que valide mal el monto y otorgue el descuento en compras bajas (Regla 2). Un buen testing tiene que asegurar que el sistema haga lo que tiene que hacer, pero también que no haga lo que NO debe hacer.

Parte 3: Pruebas Estructurales y Niveles de Prueba

1. Cobertura de Código (Code Coverage)

Es una métrica técnica de Caja Blanca que se calcula en porcentaje. Indica qué cantidad del código fuente de un programa fue realmente ejecutado por las pruebas que armamos. Nos sirve para saber si nos quedaron funciones, condicionales o bloques de código sin testear, los cuales representarían un peligro si tienen fallas ocultas.

2. Análisis de Pseudocódigo

Si usamos el valor edad = 25 para la función, el programa va a entrar por el camino del IF (porque 25 es mayor o igual a 18) e imprimirá "Es mayor". El código parece funcionar bien, pero la prueba es insuficiente porque el bloque del ELSE jamás se ejecuta.

- **Porcentaje de Cobertura de Decisión alcanzado:** 50%. Como el condicional tiene dos ramas posibles (True y False) y solo pasamos por una, cubrimos la mitad.
- **Valores para el 100% de Cobertura de Decisión:** Necesitamos dos casos de prueba. Por ejemplo, edad = 25 (para probar la rama verdadera) y edad = 12 (para obligar al programa a ir por el ELSE y probar la rama falsa).

3. Pirámide del Testing y Niveles de Prueba

La Pirámide del Testing es un modelo gráfico que nos dice cómo distribuir el esfuerzo de las pruebas en un proyecto. La lógica es que la base sea muy ancha (muchas pruebas unitarias, que son rápidas y baratas) y se vaya achicando hacia la punta (pocas pruebas de interfaz o de sistema, que son más lentas y costosas de mantener).

- **Pruebas Unitarias (Base):** Prueban la unidad de código más chica posible de forma aislada, como una sola función o método. Las hace el desarrollador mientras escribe el código (Caja Blanca).
- **Pruebas de Integración (Medio):** Verifican que los distintos módulos o componentes del sistema interactúen bien entre sí cuando se conectan (por ejemplo, probar que el sistema se comunique bien con la base de datos).
- **Pruebas de Sistema (Cima):** Evalúan el sistema completo e integrado de punta a punta (End-to-End). Se hacen simulando el comportamiento real de un usuario final (Caja Negra).

4. Pruebas Funcionales vs. No Funcionales

- **Pruebas Funcionales:** Evalúan el "qué" hace el sistema, es decir, si cumple con las acciones del negocio. *Ejemplo:* Verificar que al poner el usuario y contraseña correctos, el sistema te deje entrar al panel de control.
- **Pruebas No Funcionales:** Evalúan el "cómo" o "qué tan bien" responde el sistema (rendimiento, seguridad, usabilidad). *Ejemplo:* Hacer una prueba de estrés metiendo 2.000 usuarios en simultáneo para ver cuántos segundos tarda en cargar la página y si el servidor aguanta la carga sin caerse.

Parte 4: Aplicaciones en el Mundo Real y Preguntas para Investigar

1. Falla de software famosa: El desastre del Ariane 5 (1996)

El cohete espacial Ariane 5 se destruyó por completo a los 40 segundos de despegar en su primer vuelo. El problema fue un error de software de desbordamiento de enteros (integer overflow). Un componente de la computadora de navegación intentó convertir un valor numérico de velocidad horizontal de 64 bits (punto flotante) a un entero con signo de 16 bits. Como el número era gigantesco y no entraba en los 16 bits, el sistema tiró una excepción no controlada, las pantallas de diagnóstico fallaron y los motores se desviaron bruscamente, causando la destrucción del cohete.

Especulo que acá fallaron principalmente las **Pruebas de Integración**. El módulo de software que causó el desastre venía del modelo anterior (Ariane 4), donde funcionaba perfecto porque ese cohete manejaba velocidades menores. De forma aislada (unitaria), el módulo andaba bien. El problema fue que al integrarlo en el nuevo sistema del Ariane 5, que tenía especificaciones físicas y velocidades mucho más altas, los componentes interactuaron mal y nadie probó ese escenario integrado con datos reales del nuevo cohete.

2. Preguntas de Investigación

Pregunta 1: Selenium vs. Cypress

Selenium funciona mediante un driver externo (WebDriver) que traduce los comandos de la prueba y los manda al navegador de forma remota. Soporta casi cualquier lenguaje de programación (Java, Python, C#) y permite testear flujos complejos que abran múltiples pestañas o interactúen con diferentes dominios.

Cypress funciona de una manera totalmente distinta: se ejecuta directamente adentro del navegador, corriendo en el mismo bucle de eventos que la propia aplicación web (usa Node.js). Esto hace que sea extremadamente rápido, controla mejor los elementos de la pantalla y no necesita configurar drivers externos.

¿Cuándo usar cada uno? Es mejor usar Selenium si estás en un proyecto grande donde necesitás probar compatibilidad en navegadores viejos o flujos que requieran saltar entre pestañas y dominios externos. En cambio, si estás desarrollando una aplicación moderna (por ejemplo con React o Next.js) y querés hacer pruebas ágiles, rápidas y estables de punta a

punta en navegadores modernos, Cypress es mucho más cómodo y eficiente.

Pregunta 2: Testing de Regresión

El testing de regresión consiste en volver a correr las pruebas que ya teníamos guardadas sobre una aplicación que sufrió modificaciones, arreglos de bugs o actualizaciones de código. El objetivo de esto es asegurarnos de que los cambios nuevos no hayan roto nada de lo que ya andaba bien.

En las metodologías ágiles, el software cambia y crece todo el tiempo en ciclos muy cortos (Sprints de dos semanas). Probar a mano todo el sistema viejo en cada entrega para ver si "sigue andando" se vuelve imposible por cuestiones de tiempo. Por eso la regresión es la candidata perfecta para la automatización: permite armar una suite de pruebas fijas que se ejecutan solas en pocos minutos cada vez que un programador sube código nuevo, dándole al equipo la seguridad de que el sistema sigue estable sin frenar el ritmo de trabajo.

Fuentes consultadas:

- Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico* (7a ed.). McGraw-Hill.
- Apuntes de clase y material complementario de la Unidad 7 - UTN.
- Lions, J. L. (1996). *Ariane 5 Flight 501 Failure: Report by the Inquiry Board*.
- Documentación oficial de Cypress (cypress.io) y Selenium (selenium.dev).